Thai Poetry Rhyme Verification using a Hybrid Rule-Based Approach for Education and GenAI

1st Warit Yuvaniyama
School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
https://orcid.org/0009-0007-1630-4286

2nd Athit Phinyachok
School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
https://orcid.org/0009-0000-3783-8104

3rd Gunn Sanguankittipun
School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
https://orcid.org/0009-0005-8812-4992

4th Leo Colombo
School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
https://orcid.org/0009-0004-9133-5398

5th Prachya Boonkwan*
School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
https://orcid.org/0000-0003-1405-1071*

***Abstract*—In Thai Klon-Paed (กลอนแปด) poetry (the eight-syllable verse form) each four-line stanza must satisfy an interlocking rhyming pattern where each rhyming syllables must share both a vowel sound (sara สระ) and a final-consonant class (mattra มาตรา ตัวสะกด). Because Thai is written without word boundaries and its orthography conceals vowels and silenced letters, automatic rhyme verification is non-trivial: neither classical rule-based checkers nor G2P-romanization checkers handle the resulting edge cases reliably. We present a hybrid rule-based verification approach: first, we improve the KhaveeVerifier algorithm—vowel analysis, final-consonant-class analysis, the rhyme test, karun silencing, and true-final detection—and ship it in PyThaiNLP 5.3.5 (Model B); second, we introduce Klonpad (Model D), which augments these rules with a gold-standard G2P override dictionary of 4,292 entries and a fallback segmenter, recovering phonetic blind spots of the base oracle. We evaluate five checker configurations on a gold corpus of 36,475 classical stanzas (145,709 rhyme checks) and on a targeted augmentation of 11,623 instances including 10,000 adversarial negatives and 423 oracle-blind rhymes. The best configuration reaches 88.5% gold recall and 86.9% F1 on the augmentation, 100% precision on curated negatives, and 95.7% of the oracle-blind rhymes. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based Klon-Paed generation.**

***Index Terms*—Thai NLP, Thai poetry, Klon-Paed, rhyme verification, rule-based systems, grapheme-to-phoneme, reinforcement learning, educational technology**

I. Introduction

Thai orthography does not map straightforwardly onto pronunciation, and this gap becomes especially apparent in the classical register used by Thai poetry. The script is written without explicit word boundaries; vowels are frequently implicit; consonants or entire clusters can be silenced by the thanthakhat (ทัณฑฆาต) (karun) diacritic; and irregular words whose pronunciations do not follow the standard phonetic rules are common. Complex phonological structures, such as consonant blending (*kham khuap klam* คำควบกล้ำ) and leading consonants (*aksorn nam* อักษรนำ), further obscure syllable boundaries, vowel classification, and tone. As a result, syllabification and pronunciation are particularly difficult for computational algorithms and machine learning models.

Klon-Paed is a Thai verse form that depends on these phonetic properties. Each stanza (*bot* บท) consists of four sets of seven-to-nine-syllable lines (*wak* วรรค). A valid rhyme requires two syllables to share both the vowel sound (sara สระ) and the final-consonant class (mattra มาตราตัวสะกด). Four canonical rhyme rules must hold within (r1, r2, r3) and across (rX) stanzas (Section II), and the final syllable of every wak must participate. A single silent letter, e.g. the silent *(ร ร)* of *phet (เพชร)* ('diamond'), or a short-long vowel-length error is enough to break a rhyme. These constraints are strict and deterministic, which is difficult for statistical systems.

A reliable automatic verifier is needed in two communities. In *education*, teachers and students need deterministic, explainable, hallucination-free feedback: an LLM that invents an incorrect rhyme should not be used in a classroom. In *NLP research*, training a language model to compose Klon-Paed, e.g. via reinforcement learning (RL), requires a programmatic reward that scores rhyme correctness. Because LLMs are probabilistic, they are unreliable at strict constraints such as syllable counts and rhymes; a rule-based verifier supplies the ground truth that such generative pipelines require.

Existing verification systems fall short of this standard. The prior KhaveeVerifier in PyThaiNLP 5.0.1 (Model A) relies on

dictionary subword tokenization, omits one canonical rhyme rule, and reaches only 73.4% recall on our gold corpus. G2P romanization-based checkers, such as the `word_check` grader of KlonSuphap-LM [11] (Model C), collapse long/short vowel distinctions and produce thousands of false positives on adversarial negatives. Neither family handles the phonetic blind spots of Thai orthography in a principled way. In this paper we address these limitations with a hybrid rule-based approach and a rigorous, adversarial evaluation. Our contributions are:

- **An improved rule-based verifier (Model B).** We rework the vowel analysis (`check_sara`), the final-consonant-class analysis (`check_marttra`), the rhyme test (`is_sumpus`), karun silencing, and true-final detection, and implement all four canonical rhyme rules. The result is merged into PyThaiNLP 5.3.5 as the default `KhaveeVerifier`.
- **A dictionary-enhanced variant (Klonpad, Model D).** We augment the rules with a gold-standard 4,292-entry G2P override dictionary and a fallback segmenter, recovering the oracle's blind spots.
- **A large evaluation corpus.** 36,475 gold stanzas (145,709 rhyme checks) from five classical works, plus 11,623 augmentation instances: 10,000 oracle-verified negatives, 1,200 tricky positives, and 423 oracle-blind positives.
- **A systematic comparison.** Five configurations (A, B, C, D_w2p, D_ssg) are compared on precision, recall, F1, and coverage, with per-rule and per-operator error analyses.
- **Educational tool and RL-ready reward.** An explainable web tool plus a deterministic reward signal for RL-based Klon-Paed generation.

II. Background and Related Work

A. The Linguistic Rules of Klon-Paed

A Klon-Paed stanza has four waks of eight syllables each. Thai syllables are written without delimiters and a word may consist of one or more syllables, so the meter must be recovered by syllabification rather than by counting words. Within this skeleton the verse follows a fixed rhyme scheme. As defined by the Royal Institute of Thailand, two syllables rhyme (*sumpus* สัมผัส) when they share the same vowel sound (sara สระ) and the same final-consonant class (mattra มาตราตัวสะกด). Additionally, tone is not part of the rhyme. The mattra classes (*mae* แม่), consisting of ka, kong, kok, kot, kon, kop, kom, koey, and koew, group the final consonants of Thai syllables by their phonetic outcome (the velar, dental, labial, nasal, semivowel, and (*w* ว) finals). The four waks are called the (*sadap* สดับ), (*rap* รับ), (*rong* รอง), and (*song* ส่ง) respectively. Four canonical rhyme rules are checked in this work, following the *klon* canon and the implementation of the PyThaiNLP `KhaveeVerifier`:

- r1 (*sadap* → *rap*): the final syllable of wak 1 must rhyme with one of the first five syllables of wak 2 (syllables 3 and 5 are obligatory, 1, 2, and 4 are permissible);
- r2 (*rap* → *rong*): the final syllable of wak 2 must rhyme with the final syllable of wak 3;

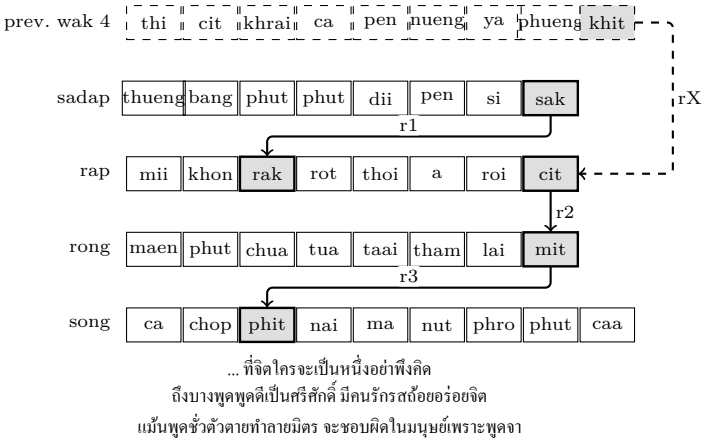

Fig. 1. The example stanza (Section 2.1) with its rhyme positions shaded and linked: r1 (sadap→rap), r2 (rap→rong), r3 (rong→song); the dashed rX arrow links the final syllable (khit) of the previous stanza's wak 4 (top row, dashed) to wak 2's final syllable (cit).

- r3 (*rong* → *song*): the final syllable of wak 3 must rhyme with one of the first five syllables of wak 4 (syllables 3 and 5 are obligatory, 1, 2, and 4 are permissible);
- rX (inter-stanza): the final syllable of wak 4 of the previous stanza rhymes with the final syllable of wak 2 of the current stanza.

As a concrete example, the well-known stanza of the *Phukao Thong* (นิราศภูเขาทอง) travel poem by Sunthorn Phu satisfies r1–r3; Figure 1 shows its rhyme positions and the inter-stanza (rX) link to the preceding stanza.

Three orthographic phenomena make these rules hard to apply automatically. First, compound, transformed, and reduced vowels (*sara prasom* สระประสม), (*sara plianrup* สระเปลี่ยนรูป), (*sara lotrup* สระลดรูป) let several spellings map to one vowel sound. Second, the thanthakhat (karun) diacritic silences a letter or a whole cluster, leaving unpronounced letters in the written form. Third, a trailing y, l, w (ย, ล, ว) may be a genuine final consonant or merely part of a vowel digraph or an initial cluster. Any verifier must resolve all three before it can compare vowel and final class.

B. Thai Text-Processing Foundations

Two building blocks underlie all Thai NLP systems: segmentation and grapheme-to-phoneme conversion. Because Thai has no word delimiters, segmentation has received sustained attention: dictionary-based maximum matching (the `newmm` engine of PyThaiNLP [1]), neural segmenters such as DeepCut [5] and AttaCut [3], syllable-based neural segmentation [4], and the SSG syllable segmenter shipped in PyThaiNLP 5.x, used throughout this work. G2P conversion maps surface orthography to pronunciation; Thai G2P has been addressed with rule-based (TLTK [6]), example-based [7], CRF joint-sequence [8], and two-stage sequence [9] models. G2P is directly relevant to rhyme verification because rhyme is a property of *sound*, not of spelling: Model C performs its comparison in the romanized space of the TLTK G2P model.

C. Computational Poetry Analysis and Generation

Prior work on Thai poetry in NLP has concentrated on analysis and generation. On the analysis side, machine-learning studies of Sunthorn Phu's verse have extracted sound patterns such as rhyme, assonance, and alliteration using internal-rhyme rules [10]. On the generation side, the dominant line of work fine-tunes language models on corpora of classical verse: the Phra Aphai Mani LM [12] and its successor KlonSuphap-LM [11] fine-tune a GPT-2 Thai base model, then use *rhyme tagging* (annotating the tokens that must rhyme), attention masking, and finally reinforcement learning in which a rule-based *klon grader* supplies the reward. That grader is the `word_check` module evaluated here as Model C, making our evaluation directly relevant to the RL step of that pipeline. More broadly, RLHF has established reward models as essential for aligning LLMs to follow strict requirements [2], and grammar-constrained decoding has been proposed to enforce structural constraints at inference time [14]; recent benchmarks report that even strong LLMs struggle with classical Thai vocabulary and verse [13], motivating dedicated deterministic tools.

D. Positioning of This Work

Compared with this landscape, our contribution is the first systematic, adversarially augmented evaluation of Thai rhyme *verification* as opposed to generation: Model A has never been stress-tested against curated edge cases, the G2P-based grader (Model C) has not been validated for precision, and no open verifier covers all four rhyme rules with dictionary support for irregular pronunciations. We fill this gap with improved rule-based verification (Model B), a dictionary-enhanced variant (Model D), and a large gold-with-augmentation evaluation, including a novel *oracle-blind* probe that holds the verifier itself accountable for rhymes its own oracle cannot see.

III. Methodology: Hybrid Verification Architecture

A. Core Rule-Based Verification (Model B)

Model B is the `KhaveeVerifier` of PyThaiNLP 5.3.5. It verifies a poem in three stages. (1) Each wak is tokenised directly into syllables with the SSG CRF segmenter, applied to the wak text itself without word pre-segmentation. (2) For each canonical rule the relevant syllables are extracted: the final syllable of every wak and the first five syllables of waks 2 and 4. (3) The rhyme test `is_sumpus` decides whether two syllables rhyme; a rule holds if the test succeeds for at least one permitted target. The eight-syllable meter is additionally reported. Algorithm 1 summarises this flow. The `check_klon` entry point splits the poem on whitespace, groups waks into stanzas of exactly four (rejecting incomplete stanzas), enforces the metre with per-wak word-count limits—ten for Klon 8 and five for Klon 4—and reports every violation as a structured message naming the stanza, the wak, and the two syllables that failed to rhyme.

The accuracy of the verifier rests on three functions. `check_sara` returns the canonical vowel of a syllable. Its per-character loop was rewritten to remove the shadowed conditions that misparsed the *ua* diphthong and to resolve the transformed and reduced vowels signalled by the short-vowel mark (*mai tai khu* ไม้ไต่คู้): *cet* → short *e*, *khaeng* → short *ae*, *ko* → (*ao* เอาะ). The *ro han* (รร) is evaluated once, yielding *am* in a *mae kom* context (e.g., *tham* (ธรรม)) and *a* elsewhere. The three readings of (*ru* ฤ) and (*lue* ฦ) are classified from the original spelling *before* karun stripping, so silent suffixes cannot hide them: *roek* in *roek* (ฤกษ์), *rit* in *rit* (ฤทธิ์) and *angkit* (อังกฤษ), and *rue-du* in *ruedu* (ฤดู). Pali/Sanskrit words with a silent terminal vowel (e.g., *kiet* (เกียรติ), *chat* (ชาติ), *thammachat* (ธรรมชาติ)) have it stripped, and an empty result falls back to the implied short *a*. `check_marttra` returns the Royal-Institute final-consonant class of a syllable, strictly by orthography. It resolves karun first, strips silent terminal vowels, then strips a silent *r* in loanword clusters—always in *tr*/*chr*/*pr* (e.g., *but*, *net*, *phet* (เพชร)) but in *kr*, *kh-r*, and *th-r* only when the preceding vowel is short, so *cak* (จักร) loses its *r* while *mangkon* (มังกร) does not. Standalone characters (*na* (ณ), *tha* (ธ), *phona* (พณ), *bo* (บ)) and the *sara koen* are classed as *mae ka*. Finally, `is_sumpus` normalizes both arguments independently, since the old conditional chain short-circuited when only one word needed normalization: *a+koey* → *ai+ka* (so *chai* rhymes with *sai*) and *a/am+kom* → *am+ka* (so *cam* rhymes with *kam*); it returns true iff the normalised sara and mattra of the two syllables are equal.

Two further mechanisms account for most of the improvement over the 5.0.1 baseline. `handle_karun_sound_silence` strips characters silenced by the thanthakhat, no longer by a fixed truncation: it recognises multi-letter silent suffixes such as *phra lak* (พระลักษมณ์) (strips the silent *km* cluster to expose the true *mae kok* final) and *kasat* (กษัตริย์) (strips the silent *triy* suffix, leaving a *mae kot* final), two-consonant suffixes such as *-tr* in *sat* (ศาสตร์) and *-nthr* in *can* (จันทร์), and the consonant-plus-vowel patterns of *sit* (สิทธิ์) and *phan* (พันธุ์), while leaving an internal karun (e.g., *ohm* (โอห์ม)) untouched. `_is_true_final` decides whether a trailing *y*, *l*, or *w* is a genuine final rather than part of a vowel digraph or initial cluster. It strips karun and tone

Algorithm 1 Rhyme verification of a Klon-Paed poem (`check_klon`).

Require: poem T with waks separated by whitespace; type $k \in \{4, 8\}$
Ensure: list of rhyme errors (empty $\Leftrightarrow$ poem is valid)

```
W ← split T into waks, grouped into stanzas of four
for each stanza (w_1, w_2, w_3, w_4) do
    S_i ← SSG(w_i), i = 1, ..., 4 {syllables}
    for each rule ρ ∈ {r1, r2, r3, rX} do
        (a, B_ρ) ← source syllable and target set of rule ρ
        if no b ∈ B_ρ satisfies IS_SUMPUS(a, b) then
            record an error for rule ρ
        end if
    end for
end for
return the list of errors
```

TABLE I
PRINCIPAL CHANGES OF THE 5.3.5 KHAVEEVERIFIER OVER THE 5.0.1 BASELINE.

Aspect	5.0.1 → 5.3.5 (example)
Segmentation	dict subword → SSG syllable segmentation
Karun (thanthakhat)	ignored → silenced (ohm→o)
True finals y/l/w	naive → cluster-aware (thai, klai→mae ka)
Special-vowel norm.	one-sided → both-sided (cam∼kam)
Compound/ reduced vowels	misread → compound-vowel classifier, short-vowel mark, and ua diphthong (cet→short e; tua∼chua)
ru	one sound → three sounds (roek, rit, rue-du)
r3 rule	absent → implemented (all four rules)
Silent r	misread → stripped in `check_marttra` (but, phet)
Single-char words	crash → guarded (na, tha→a)
Silent terminal vowels	misread → stripped (kiet, chat)
Stanza processing	flat → stanza-based, word counts, structured errors

marks first—so *klai* (ไกล้), which ends in a tone mark, is still seen as ending in *l*—requires at least two consonants, and consults hand-built cluster tables for *l*, *r*, and *w* under the pre-posed vowels (*e-*, *ae-*, *o-*, *ai-* เ-แ-โ-ไ-): the *y* of *thai* (ไทย) is silent, the *l* of *plae* (แปล) belongs to the cluster, and the *l* of *chon* (ชล) is a true final.

Table I summarises the principal changes of the 5.3.5 verifier relative to the 5.0.1 baseline; they were developed and merged in the PyThaiNLP project [1]. The fixes target precisely the failure families probed by the augmentation (Section V): karun and *ru* traps, short/long and compound-vowel swaps, and the one-sided special-vowel normalization that our C9 operator exploits. Together they raise per-rule gold recall from 86–89% to 95–97% (5.3.5) and eliminate false positives on the curated negatives.

B. Dictionary-Enhanced Verification (Klonpad, Model D)

Model B is linguistically principled but inherits a blind spot from its own design: `check_sara` returns only the first vowel of a token, so a multi-syllable word that SSG keeps as one token loses the vowel of its final (rhyming) syllable, and silent-*r* loanwords are stripped only in a whitelisted subset of clusters. Klonpad (Model D) addresses this by layering a gold-standard pronunciation dictionary on top of Model B's rules.

The dictionary, `POETRY_OVERRIDES`, contains 4,292 hand-verified entries mapping a surface form to its syllable sequence (e.g., *phet* (เพชร) → *phet*; *sattruu* (ศัตรู) → *sat-truu*; *kasatrii* (กษัตรี) → *ka-sat-trii*). It was built from the frequent vocabulary of the Phra Aphai Mani corpus with manual supervision, and every entry is checked by a four-guard pipeline that rejects pronunciations hallucinated by the Word2Phrase (w2p) neural G2P model; w2p-derived candidates are kept only as labelled hints and never trusted on their own. At inference time Klonpad segments the wak into words with an override-augmented newmm dictionary, looks each up in the override dictionary, and syllabifies unmatched words with the fallback segmenter. We report two configurations: `D_w2p`, which falls back to the w2p neural G2P model, and `D_ssg`, which falls back to plain SSG segmentation without w2p; as Section V shows, the w2p fallback introduces pronunciation hallucinations on short or compound words, and `D_ssg` is the more robust configuration.

C. Baseline Systems

Model A is the original `KhaveeVerifier` of PyThaiNLP 5.0.1, instrumented verbatim from the vendored source: dictionary-based `subword_tokenize`, no r3 rule, no karun handling, no true-final logic. Model C is the `word_check` rhyme grader of KlonSuphap-LM [11]: each wak is romanized with the TLTK G2P model and the romanized vowels and finals compared. Its two documented weaknesses are (i) a `replace_long_short` step that collapses long and short vowels and (ii) a mattra comparison that ignores the final consonant, both inflating recall at the cost of precision; tltk failures reduce coverage.

IV. DATASET CONSTRUCTION AND AUGMENTATION

A. Gold-Standard Corpus Compilation

We collected five classical Thai narrative poems from the Vajirayana digital library [15]: *Khobut* (โคบุตร), *Khun Chang Khun Phaen* (ขุนช้างขุนแผน), *Phra Aphai Mani* (พระอภัยมณี), *Phukao Tong* (ภูเขาทอง), and *Suphasaet Son Ying* (สุภาษิต สอน หญิง). A cleaning pipeline normalised the text and split it into stanzas of four waks, discarding only chapter-opening fragments and scribal *o* marks. Crucially, waks with ten or more syllables were retained: dropping them severs inter-stanza rhyme chains and made an early corpus version appear to violate rX at a 72% rate, whereas on the complete corpus rX holds at 96–97%. The final corpus comprises 191 files, 36,475 stanzas, 145,900 waks, and 145,709 gold rhyme checks; the gold is all-positive, because the classical poems are presumed to rhyme. An alternative public corpus covering similar material is the Thai Literature Corpora [16].

B. Targeted Data Augmentation

Recall over an all-positive corpus cannot measure precision, and easy positives cannot expose blind spots. We therefore augmented the gold with adversarial instances generated by an automated pipeline. Every instance is *standalone*: it is derived from a real stanza but only the target rule is changed, so exactly one rule is exercised while the others remain intact. Every negative is verified by the 5.3.5 oracle (`is_sumpus`) to be a genuine non-rhyme, and the whole set was audited in four independent passes in which reviewers spelled out the sara and mattra of every candidate (with vowel length made explicit) and double-checked pronunciations against the IPA. The audit removed 56 candidate words the oracle misreads—karun clusters it fails to silence and multi-syllable Pali words SSG keeps as single tokens.

a) Negatives (precision probes).: The 10,000 negatives are drawn from eight corruption operators: trivial different-vowel-and-mattra swaps (C6, 300), same-mattra swaps (C0, 500), same-vowel swaps (C1, 500), short/long vowel swaps (C3, 3,100), liquid-final swaps (C4, 1,200), a systematic trap in which the 5.0.1 checker reads the candidate as rhyming with the target while 5.3.5 does not (C9, 2,800), leading-consonant words (C5, 600), and karun/*ru* (ฤ)/cluster traps (C2, 1,000). C3 and C9 dominate the mix because they are the operators that best discriminate the checkers.

b) Positives (recall probes).: The 1,200 *tricky* positives swap the target syllable with a rhyming candidate that exercises normalization (*sara plianrup/lotrup* สระเปลี่ยนรูป/ลดรูป), *ru* (ฤ), karun, and true-final cases; all are oracle-confirmed rhymes. A further 423 *oracle-blind* positives are genuine rhymes the oracle labels as non-rhymes: the silent-*r phet* (เพชร) (true *eh*+kot, which the oracle reads with the long *ae* because the dead final carries no short-vowel mark), and the first-sara words *sattruu* (สัตรู), *kasatrii* (กษัตรี), and *kasatriiya* (กษัตรีย์) (SSG keeps each as one token, so `check_sara` hears only the first vowel). For these the gold label is the *linguistic* truth, not the oracle verdict: a checker that hears the real rhyme receives credit, while the oracle itself (Model B) is charged a false negative—"the oracle that is wrong loses points."

V. Evaluation and Results

A. Experimental Setup

We report precision, recall, and F1 with Wilson 95% confidence intervals, coverage, and a conservative drop-as-fail variant for Model C. Because every augmentation instance is standalone and tests exactly one rule, the per-rule prediction is the clean metric; a stanza-level aggregate over the augmentation is only well-defined for Model B (the oracle). Model A has no r3 and is reported as N/A there. All checkers were instrumented from verbatim vendored sources (the 5.0.1 and 5.3.5 cores), with Model C run under Python 3.12 through a persistent pool of ten tltk workers (the full augmentation pass takes about 40 minutes; A/B/D are parallelised and finish in under a minute). Every number is recomputed from the committed result files.

B. Gold-Corpus Recall

Table II reports stanza-level and per-rule recall over the 36,475 gold stanzas. The proposed family dominates: D_ssg reaches 88.5% stanza recall and B 87.5%, versus 73.4% for Model A. Model C reaches 84.8% but evaluates only 95.7% of the corpus (tltk failures concentrate in the archaic Khun Chang Khun Phaen).

C. Augmentation: Precision, Recall, and F1

Table III summarises the augmentation-only results (pooled per rule). Three results stand out.

First, Model B is a perfect oracle on the negatives: zero false positives across all 10,000 (100% precision by construction, since the negatives were oracle-verified). Its recall is limited only by the oracle-blind positives it cannot see (73.9%).

TABLE II
Gold recall (stanza level and per rule), 36,475 stanzas.

System	Stanza	r1	r2	r3	rX
A (5.0.1)	73.4	86.3	88.8	–	88.7
B (5.3.5)	87.5	94.7	96.5	95.5	96.7
D_w2p	87.4	95.4	96.3	94.9	96.5
D_ssg	**88.5**	95.9	96.7	95.2	97.0
C (Kongfha)	84.8†	93.2	96.5	94.1	96.6

† coverage 95.7%; per-rule figures on the evaluated subset.

TABLE III
Augmentation-only results (pooled per rule, 11,623 instances). FP = false positives on 10,000 negatives; FN-pos = combined false negatives on 1,623 positives.

System	P	R	F1	FP	FN-pos
A (5.0.1)	28.4	62.7	39.1	1,950	848
B (5.3.5)	**100.0**	73.9	85.0	**0**	423
D_w2p	83.4	**92.1**	**87.5**	298	128
D_ssg	86.8	87.0	86.9	215	211
C (Kongfha)	30.4	87.1	45.1	3,107	266

Second, Model D recovers most of those blind spots: D_w2p reaches 92.1% recall and D_ssg 87.0%, with precision 83.4% and 86.8%, giving the best augmentation F1 (87.5% for D_w2p). Table IV shows the mechanism: D_w2p hears 405 of the 423 oracle-blind rhymes (95.7%), D_ssg 288 (68.1%), B none.

Third, the baselines invert the precision/recall trade-off. Model C is recall-strong (87.1% on the augmentation, 78.7% even on oracle-blind rhymes, because its G2P romanization hears the real sound) but makes 3,107 false positives —2,702 on the short/long vowel swap (C3), which its `replace_long_short` collapses —for 30.4% precision. Model A is weak on both sides (28.4% precision, 62.7% recall), with 1,774 of 1,950 false positives from the C9 old-accept trap. Per rule, B keeps 100% precision on all four rules (r1 recall lowest at 54.5%, rX highest at 98.0%), C's precision collapses on every rule (24–35%), and D_ssg stays above 75% precision and 82% recall throughout.

D. Oracle-Blind Probe and Error Analysis

Table IV reports the oracle-blind probe with the combined false-negative accounting (tricky + oracle-blind) and the FP breakdown by the two most informative operators. The combined FN-pos column is the headline: B carries 423 false negatives (all oracle-blind), A carries 848, and D carries only 128 (D_w2p) or 211 (D_ssg). C's precision collapse concentrates on the short/long vowel swap (C3: 2,702 of 3,107 FPs) and A's on the old-accept trap (C9: 1,774 of 1,950).

E. Discussion

The experiment supports four findings. (1) **The proposed rules are precise.** B never false-accepts on oracle-verified negatives, and B, D_w2p, and D_ssg all exceed 99.5% merged precision (A 95%, C 92%). (2) **The override dictionary recovers the oracle's blind spots.** The D_w2p-versus-D_ssg

TABLE IV
ORACLE-BLIND PROBE AND ERROR ANALYSIS (FN-POS = TRICKY + ORACLE-BLIND).

	recall (%)		FN-pos		FP	
System	tricky	oracle-blind	total	=ob	total	C3/C9
A	57.0	21.5	848	332	1,950	162/1,774
B	**100**	0.0	423	423	**0**	0/0
D_w2p	90.8	**95.7**	**128**	18	298	135/103
D_ssg	93.7	68.1	211	135	215	119/48
C	85.3	78.7	266	90	3,107	2,702/267

gap on oracle-blind recall (95.7% versus 68.1%) is the pronunciation-knowledge contribution: the overrides read *phet* as /phet/ and split *sattruu* and *kasatrii* into their true syllables. (3) **C trades precision for recall on exactly the vowel-length distinction** the rule-based family enforces (78.7% oracle-blind recall, but 30.4% precision). (4) **The oracle itself has a blind spot** that only the oracle-blind probe exposes: a purely oracle-supervised evaluation would report B as flawless. On the merged gold+augmentation set, F1 favours D_ssg (93.4%), D_w2p (92.9%), and B (92.8%), with C (88.0%) and A (82.3%) behind; the discriminating precision lives in the augmentation-only tables.

VI. Applications and Future Work

A. Educational Impact

The verifier is deployed as an interactive web tool for Klon 4/8 teachers and students: it segments a poem into syllables, highlights rhyme positions, spells out the sara and mattra of every syllable, and explains each error in terms of the four rhyme rules. Because it is rule-based, the feedback is reproducible and free of hallucination—properties general-purpose LLMs cannot guarantee. Future work includes classroom evaluations and pronunciation guidance for irregular words.

B. Foundation for Generative AI

The same determinism makes the verifier a suitable reward signal for fine-tuning small, locally run LLMs: exact, immediate, and free of the noise of a learned reward model. This mirrors the RL step of the KlonSuphap-LM pipeline [11], but with a grader (Model B or D) substantially more precise than the G2P-based grader used there (Model C), particularly on vowel length. Fine-tuning a generator with this reward, scaling the override dictionary, and context-sensitive pronunciations are future work, alongside decoding-time constrained sampling [14].

VII. Conclusion

We presented a hybrid rule-based approach to Thai Klon-Paed rhyme verification. An improved rule-based verifier (Model B), now the default `KhaveeVerifier` of PyThaiNLP 5.3.5, implements all four canonical rhyme rules with robust karun, true-final, and sara handling and reaches 100% precision on curated negatives. A dictionary-enhanced variant (Klonpad, Model D) adds a gold-standard 4,292-entry G2P override dictionary and recovers 95.7% of the oracle's own blind spots. On 36,475 gold stanzas and 11,623 adversarial instances, the proposed family raises stanza recall from 73.4% to 88.5% (D_ssg) with near-perfect precision, exposing the blind spots of both the G2P-romanization checker and the rule-based oracle. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based generation; its integration into PyThaiNLP makes the improvement available to the entire Thai NLP ecosystem.

Acknowledgment

We thank the Vajirayana Digital Library [15], the PyThaiNLP maintainers for merging the `KhaveeVerifier` improvements, and the TLTK and KlonSuphap-LM developers.

References

[1] PyThaiNLP Contributors, "PyThaiNLP: Thai natural language processing in Python," 2024. [Online]. Available: https://github.com/PyThaiNLP/pythainlp

[2] L. Ouyang, J. Wu, X. Jiang, et al., "Training language models to follow instructions with human feedback," in NeurIPS, 2022.

[3] P. Chormai, P. Prasertsom, and A. T. Rutherford, "AttaCut: A fast and accurate neural Thai word segmenter," arXiv preprint arXiv:1911.07056, 2019.

[4] P. Chormai, P. Prasertsom, J. Cheevaprawatdomrong, and A. T. Rutherford, "Syllable-based neural Thai word segmentation," in Proc. 28th International Conference on Computational Linguistics (COLING), 2020.

[5] T. Kittinaradorn, et al., "DeepCut: A Thai word tokenization library using deep neural network," 2019. [Online]. Available: https://github.com/rkcosmos/deepcut

[6] W. Aroonmanakun, "TLTK: Thai language toolkit," [Online]. Available: https://pypi.org/project/tltk/

[7] P. Charoenpornsawat and T. Schultz, "Example-based grapheme-to-phoneme conversion for Thai," in Proc. Interspeech, 2006.

[8] S. Saychum, S. Kongyoung, A. Rugchatjaroen, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient Thai grapheme-to-phoneme conversion using CRF-based joint sequence modeling," in Proc. Interspeech, 2016.

[9] A. Rugchatjaroen, S. Saychum, S. Kongyoung, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient two-stage processing for joint sequence model-based Thai grapheme-to-phoneme conversion," Speech Communication, 2019.

[10] P. Suksanguan, S. Waijanya, and N. Promrit, "The extraction of beautiful sound patterns from Sunthorn Phu's poem using machine learning technique and internal rhyme rule," 2021.

[11] K. Yingyingseree, "KlonSuphap-LM: GPT-2 for Thai Klon-Paed," Hugging Face model, 2024. [Online]. Available: https://huggingface.co/Kongfha/KlonSuphap-LM

[12] K. Yingyingseree, "PhraAphaiManee-LM," Hugging Face model, 2023. [Online]. Available: https://huggingface.co/Kongfha/PhraAphaiManee-LM

[13] T. Triyason, "Lilit to Ramakien: A five-work benchmark for LLM understanding of classical Thai literature," in Proc. ICAIT, ACM, 2026.

[14] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without fine-tuning," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[15] Vajirayana Digital Library. [Online]. Available: https://vajirayana.org

[16] A. Rutherford, "Thai literature corpora (TLC)," [Online]. Available: https://attapol.github.io/tlc.html